

Cloud Provider Analytics

Primer Parcial - Diseno preliminar

Fecha: 2026-04-16

Proyecto: Cloud Provider Analytics

Curso: Big Data (Primer Cuatrimestre 2026)

Documento de diseno preliminar orientado a validar viabilidad tecnica temprana y dejar una base directa para la implementacion del MVP tecnico (segundo parcial) y la expansion de alcance en la entrega final.

Resumen ejecutivo

- Arquitectura seleccionada: Lambda (batch + streaming).
- Almacenamiento intermedio: Parquet por zonas Landing/Bronze/Silver/Gold.
- Serving analítico: AstraDB/Cassandra con modelado query-first.
- Objetivo del primer parcial: validar diseño y plan de ejecución, evitando sobre-ingeniería.

Contexto

Como equipo de datos de un cloud provider, necesitamos cubrir dos necesidades en paralelo: - Near real-time para métricas operativas de uso/costo. - Batch diario o mensual para maestros y facturación.

Los datos llegan con nulos, inconsistencias, duplicados y evolución de schema (v1/v2), por lo que la arquitectura debe priorizar robustez e idempotencia sin sobre-complicar la primera fase.

Objetivo de esta entrega

Presentar un diseño preliminar claro, visual y accionable para validar viabilidad técnica temprana. El alcance se mantiene conciso (evitando over-engineering), pero dejando base directa para el MVP técnico del segundo parcial y el cierre del final.

1) Diagrama de arquitectura de alto nivel

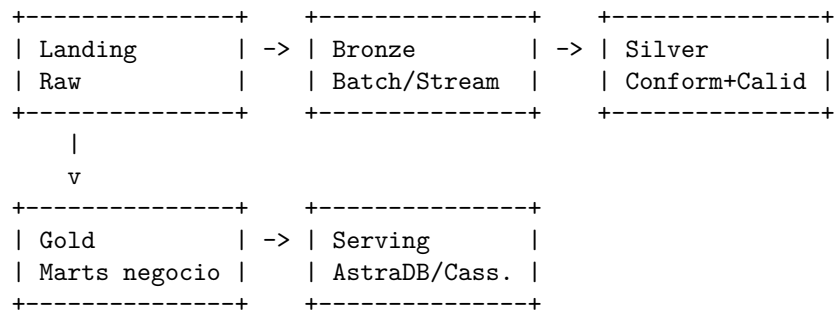
Diagrama por capas y flujo principal:

Landing (Raw immutable) -> Bronze (Batch + Streaming) -> Silver (Conformado + Calidad) -> Gold (Marts de negocio) -> Serving (AstraDB/Cassandra) -> BI/Visualización

Capa	Componentes principales	Salida
Landing	customers_orgs, users, resources, support_tickets, marketing_touches, nps_surveys, billing_monthly, usage_events_stream	Archivos crudos inmutables
Bronze	Ingesta batch + ingesta streaming con watermark, dedupe y checkpoint	Parquet estandarizado + trazabilidad técnica

Capa	Componentes principales	Salida
Silver	Limpieza, tipificacion, joins, reglas de calidad y quarantine, compatibilidad schema v1/v2	Datos conformados listos para analitica
Gold	Marts FinOps, Soporte y Producto/GenAI	Tablas orientadas a negocio
Serving	AstraDB/Cassandra modelado query-first	Consultas CQL para dashboards

Vista visual (alto nivel)



Diagramas por capa (paginas dedicadas)

Capa 1 - Landing

Fuentes de entrada

- | - customers_orgs.csv
- | - users.csv
- | - resources.csv
- | - support_tickets.csv
- | - marketing_touches.csv
- | - nps_surveys.csv
- | - billing_monthly.csv
- | - usage_events_stream/*.jsonl

Salida: archivos crudos inmutables (sin transformacion)

Objetivo de la capa: - Mantener una referencia fiel de origen para auditoria y reproceso.

Capa 2 - Bronze

Landing

```
|--(Batch ingest)-----> Bronze Batch (Parquet)
|--(Structured Streaming ingest)-----> Bronze Stream (Parquet)
```

Controles base:

- Schema explicito
- ingest_ts / source_file
- watermark + dedupe event_id (stream)
- checkpoint

Objetivo de la capa: - Estandarizar estructura minima y asegurar trazabilidad tecnica temprana.

Capa 3 - Silver

Bronze Batch + Bronze Stream

|

---> Limpieza y tipificacion

---> Conformance y joins con maestros

---> Reglas de calidad + quarantine

---> Compatibilizacion schema v1/v2

v

Silver conformado

Objetivo de la capa: - Convertir datos crudos en datos analiticamente confiables.

Capa 4 - Gold

Silver

- | - Mart FinOps (org_daily_usage_by_service)
- | - Mart Soporte
- | - Mart Producto/GenAI

Salida: datasets orientados a consulta de negocio

Objetivo de la capa: - Exponer metricas y KPIs listos para consumo analitico.

Capa 5 - Serving

Gold -> AstraDB/Cassandra (query-first)

|
+--> Consultas CQL
+--> Dashboards BI

Objetivo de la capa: - Responder consultas operativas y ejecutivas con baja latencia.

Patron elegido: **Lambda** - Batch para maestros/facturacion/encuestas. - Streaming para eventos de uso en near real-time. - Esta decision cumple lo obligatorio del MVP y deja camino directo a la entrega final.

Justificacion del patron: - Evita forzar todo a stream (Kappa) cuando parte de los datos son naturalmente batch. - Reduce complejidad inicial en Colab, sin perder escalabilidad conceptual para etapas futuras. - Permite convergencia en Gold y Serving con una vista unificada de negocio.

2) Mapeo de requisitos a componentes

Requisito clave	Componente / tecnologia	Justificacion	Etapas de validacion
Near real-time operativo	Spark Structured Streaming (Bronze stream)	Latencia baja con micro-batches y control de late data	Parcial 2
Batch de maestros/facturacion	Spark batch + Parquet particionado (Bronze batch)	Eficiente para datos periodicos y reproceso controlado	Parcial 2
Calidad de datos	Reglas + flags + quarantine en Silver	Aisla datos invalidos sin frenar el pipeline	Parcial 2 y Final
Evolucion de schema v1/v2	Compatibilizacion en Silver + contratos de schema	Reduce roturas por drift y habilita continuidad analitica	Parcial 2 y Final
Enriquecimiento y features	Joins con org/users/resources + features de negocio	Prepara marts para FinOps, Soporte y Producto	Parcial 2 y Final
Idempotencia	Checkpointing + keys naturales + upserts	Reejecucion sin duplicados y consistencia en serving	Parcial 2 y Final

Requisito clave	Componente / tecnologia	Justificacion	Etapas de validacion
Serving para consultas	AstraDB/Cassandra (query-first)	Baja latencia y modelo alineado a preguntas de negocio	Parcial 2 y Final
Performance	partitionBy + reparti- tion/coalesce + reparquet cuando aplique	Control de costo/tiempo sobre Colab y futura escalabilidad	Final
Demo ejecutiva	Consultas CQL sobre marts Gold	Evidencia funcional para dashboarding y presentacion	Final (2 minimas en Parcial 2)

Inclusion explicita de 5Vs de Big Data

V	Aplicacion en el proyecto
Volumen	Eventos de uso continuos + maestros historicos en Parquet particionado
Velocidad	Ingesta streaming con micro-batches y watermark
Variedad	CSV + JSONL + evolucion de schema (v1/v2)
Veracidad	Reglas de calidad, quarantine y trazabilidad de errores
Valor	Marts de FinOps, Soporte y Producto para decisiones operativas

3) Flujo de datos (data pipeline)

Flujo operativo por path: 1. Path batch: Landing (maestros/facturacion) -> Bronze batch -> Silver conformado. 2. Path stream: Landing (usage events) -> Bronze stream -> Silver conformado. 3. Convergencia: Silver -> Gold (marts por dominio). 4. Serving: Gold -> AstraDB/Cassandra -> consultas CQL y dashboards.

Modos declarados: - Batch: maestros y facturacion (periodico diario/mensual). - Streaming: usage_events_stream con dedupe por event_id y manejo de late data.

Herramientas por paso: - Landing -> Bronze batch: PySpark (DataFrame batch) en Colab. - Landing -> Bronze streaming: PySpark Structured Streaming. -

Bronze -> Silver: transformaciones Spark SQL/DataFrame + reglas de calidad.
 - Silver -> Gold: agregaciones Spark por mart. - Gold -> Serving: Spark -> AstraDB/Cassandra (conector o foreachBatch). - Serving -> consumo: CQL + BI (Tableau/PowerBI/Superset/Grafana).

Resultado esperado del flujo: - Misma capa Gold alimentada por ambos paths, evitando silos entre historico y near real-time. - Base tecnica lista para ampliar de FinOps minimo (Parcial 2) a cobertura completa de negocio (Final).

4) Asunciones y riesgos iniciales

Asunciones:

Asuncion	Razonable para esta etapa	Mitigacion si no se cumple
Volumen inicial manejable en Colab	Permite MVP sin infraestructura adicional	Particionado agresivo y escalado a entorno administrado
event_id util para dedupe	Habilita idempotencia temprana	Reglas de unicidad + monitoreo de colisiones
Convivencia schema v1/v2	Evolucion esperada en eventos	Contratos de schema + compatibilizacion en Silver
AstraDB disponible para serving	Necesario para demostrar query-first	Validacion temprana de keyspace/tabla y plan de fallback local

Riesgos:

Riesgo	Probabilidad	Impacto	Mitigacion
OOM en Colab por crecimiento de datos	Media	Alta	Procesamiento incremental, coalesce/repartition y ajuste de particiones
Late data mayor al watermark	Baja	Media	Ajustar watermark y auditar dropped events
Drift de schema no detectado a tiempo	Media	Alta	Alertas y validaciones de schema por capa

Riesgo	Probabilidad	Impacto	Mitigacion
Duplicados en serving por fallas de idempotencia	Media	Alta	Checkpointing estricto + upserts por clave natural
Hot partitions en Cassandra	Media	Media/Alta	Revisar cardinalidad y claves de particion por consulta
Acople alto de notebooks y logica	Media	Media	Modularizar transformaciones en componentes reutilizables

5) Estimacion de esfuerzo y recursos (rough estimate)

Supuesto de equipo: 3 personas.

Bloque	Tiempo estimado	Roles principales
Diseño y base tecnica (esta entrega)	3 a 4 dias	Data Architect + Data Engineer
Ingesta Bronze batch/stream (MVP)	4 a 6 dias	Data Engineer Batch + Data Engineer Streaming
Silver (calidad + enriquecimiento)	4 a 6 dias	Data Engineer + Analytics Engineer
Gold minimo + Cassandra (Parcial 2)	3 a 5 dias	Data Engineer + Data Modeler
Expansion de marts + optimizacion (Final)	6 a 9 dias	Data Engineer + Analytics Engineer
Storytelling, presentacion y demo final	3 a 4 dias	Todo el equipo

Recursos minimos: - PySpark en Google Colab. - Almacenamiento en Parquet por zonas (Bronze/Silver/Gold + quarantine/checkpoints). - AstraDB/Cassandra para serving. - Herramienta de visualizacion para demo.

Costo estimado de referencia: - Escenario base: costo cercano a cero usando free tiers. - Escenario extendido: costo mensual bajo si se requiere Colab Pro y/o mayor almacenamiento.

6) Decisiones para facilitar entregas futuras

- Estandarizar nomenclatura de datasets y particiones desde Bronze.
- Mantener contratos de schema por capa (landing/bronze/silver/gold).
- Diseñar Gold por preguntas de negocio (query-first) para simplificar Cassandra.
- Construir trazabilidad de calidad desde el MVP (evita retrabajo en final).
- Documentar decisiones y trade-offs por iteracion para reutilizar en presentacion/video.